



SIMATS
ENGINEERING

SIMATS Online Programming Lab
JAVA PROGRAMMING

8 SRIPATHI MARAM
SRIPATHI

CO2-AT2-Scenario Based

View Only

QUESTIONS

Q1

Smart Hospital Patient Registration System

20 marks

Q2

Online (Deep Band) Management System

20 marks

2/2 answered

Smart Hospital Patient Registration System

20 marks

Smart Hospital Patient Registration System.

A multi-specialty hospital wants to digitize its outpatient registration system.

Every patient arriving at the hospital receives a registration number. The hospital management wants a Java application that maintains patient records efficiently while preventing duplicate registrations.

Each patient record consists of

- [[Patient ID
- [[Patient Name
- [[Mobile Number
- [[Department
- [[Doctor Name

The system should utilize Java Collection Framework and String manipulation techniques for storing, validating, searching, and displaying patient information.

Functional Requirements

Develop a Java program using ArrayList and HashMap to perform the following operations:

Menu

1. Register Patient
2. Search Patient
3. Display All Patients
4. Count Department-wise Patients
5. Exit

Requirement 1 - Register Patient (4 Marks)

Accept

Patient ID

Patient Name

Mobile Number

Department

Doctor Name

Perform validation

Your Code

```
1 import java.util.*;
2
3 class Patient
4 {
5     String id;
6     String name;
7     String mobile;
8     String department;
9     String doctor;
10
11
12     Patient(String id, String name, String mobile, String department,
13         String doctor)
14     {
15         this.id=id;
16         this.name=properCase(name.trim());
17         this.mobile= mobile.trim();
18         this.department= department.trim().toUpperCase();
19         this.doctor= doctor.trim();
20     }
21
22     public static String properCase(String str)
23     {
24         str= str.trim().toLowerCase();
25         String words[] = str.split("\\s");
26         StringBuilder sb= new StringBuilder();
27
28         for(String w: words)
29         {
30             sb.append(Character.toUpperCase(w.charAt(0))).append(w.substring(1));
31         }
32
33         return sb.toString().trim();
34     }
35
36     public static void main(String[] args)
37     {
38         while(true)
```

Input

```
1
2101
Rahul Kumar
9876543210
```

Output

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

```
83
84     String name= sc.nextLine().trim();
85
86     if(!name.matches("\\d{10}"))
87     {
88         System.out.println("Invalid Patient Name");
89         return;
90     }
91
92     String mobile= sc.nextLine().trim();
93     if(!mobile.matches("\\d{10}"))
94     {
95         System.out.println("INvalid Mobile Number");
96         return;
97     }
98
99     String department= sc.nextLine().trim().toUpperCase();
100    String doctor= sc.nextLine().trim();
101
102    Patient p= new Patient(id, name, mobile, department, doctor);
103
104    patients.add(p);
105    patientMap.put(id,p);
106
107    System.out.println("Patient Registered Successfully");
108 }
109
110 public static void searchPatient()
111 {
112     String id= sc.nextLine().trim();
113
114     if(!patientMap.containsKey(id))
115     {
116         System.out.println("Patient Not Found");
117         return;
118     }
119 }
```

Q2. Online Library Book Management System

20 marks

Scenario

Online Library Book Management System

A university library is planning to automate its book management process. The librarian wants a Java application that stores book details, prevents duplicate book entries, allows students to search for books quickly, and generates category-wise reports.

The application should use Java String handling and Java Collection Framework to efficiently manage the library records.

Requirements

Each book contains the following details:

IDBook ID

IDBook Title

IDAuthor Name

IDCategory

IDPublisher

Use:

IDArrayList to store all book objects.

IDHashMap to store Book ID as the key for quick searching.

IDHashSet to maintain unique author names.

Menu Driven Program

1. Add Book
2. Search Book by ID
3. Display All Books
4. Display Unique Authors
5. Category-wise Book Count
6. Exit

Requirement 1 – Add Book (4 Marks)

Accept

Book ID

Book Title

Book Author

Book Category

Book Publisher

Use HashSet for maintaining unique authors.

Your Code

```
1 import java.util.*;
2
3 class Book {
4     String id, title, author, category, publisher;
5
6     Book(String id, String title, String author, String category, String publisher) {
7         this.id = id.trim();
8         this.title = toTitleCase(title.trim());
9         this.author = author.trim();
10        this.category = category.trim().toUpperCase();
11        this.publisher = publisher.trim();
12    }
13
14    static String toTitleCase(String str) {
15        String words[] = str.toLowerCase().split("\\s+");
16        String result = "";
17        for (String w : words) {
18            result += Character.toUpperCase(w.charAt(0)) + w.substring(1).toLowerCase() + " ";
19        }
20        return result.trim();
21    }
22 }
23
24
25 public class Main {
26
27     static Scanner sc = new Scanner(System.in);
28
29     static ArrayList<Book> books = new ArrayList<>();
30     static HashMap<String, Book> map = new HashMap<>();
31     static HashSet<String> authors = new HashSet<>();
32
33 }
```

Input

```
1
8101
Java Programming
Herbert Schildt
```

Output

Visible Test Cases

```
Test Case 1
Test Case 2
Test Case 3
```